CDC Replay Correctness Lab: A Demonstration of Ordering, Recovery, and Finite Identity Retention

Author details pending review

SIGMOD 2027 Demonstrations target — review draft, not submitted

Abstract

Change-data capture replay can produce stable but incorrect materialized state. We present a local, synthetic command-line lab comparing arrival-order replay, event-ID deduplication alone, and version-aware replay. A deterministic stale-delete counterexample exposes an ordering failure that identity deduplication cannot prevent. An independently implemented batch oracle makes errors visible. A separate finite-ID retention experiment distinguishes final-state containment from duplicate recognition and conflict detection. In a registered local run, each baseline had 30 incorrect states among 36 valid observations; version-aware replay matched the oracle in those 36 observations. These findings concern this bounded synthetic model only.

1 Motivation and demonstration scope

A consumer may receive an old delete after a newer reinsert, or reprocess events following recovery. Removing duplicate event IDs does not order distinct changes for the same key. Conversely, a replay that leaves state unchanged can merely preserve an earlier mistake. Our demonstration makes these distinctions inspectable without credentials, proprietary data, or an external service.

The modeled event has an immutable ID, key, positive integer per-key version, string value and deletion flag. A higher version defines the intended state; tombstones remain materialized. IDs and versions must come from an authoritative source. Real CDC systems may require transaction position, partition identity or an epoch to make versions comparable; this lab does not infer them.

The artifact extends an existing seeded recovery demo with two explicit baselines, controlled workloads, retained machine-readable results and a finite-ledger question. It does not introduce a new distributed replay algorithm. Its contribution is a reviewable diagnostic and teaching experiment; novelty relative to existing labs requires further review. The prior V4/V5 study examined composed reliability controls. This demo studies CDC materialization and bounded identity memory; prior research artifacts and the submitted Paper 1 are not edited.

2 Visible counterexample

Deliver a live key x at version 4 under ID a, then a deletion at version 3 under ID b. Arrival-order replay ends deleted. ID-only replay also ends deleted because both IDs are new. Version-aware replay retains live version 4. The oracle selects version 4 independently of arrival order. A unit test asserts this result without searching seeds. A stable second replay is insufficient to distinguish the correct and incorrect states. This example anchors the audience interaction rather than suggesting universal superiority of a particular storage architecture.

3 Architecture and semantics

The system separates five roles: deterministic canonical generation; delivery/failure construction; matched processors; an independent batch oracle; and artifact scoring. The data path is

$$\begin{aligned} &\text{canonical events} \to \text{controlled delivery} \\ &\to \text{three replay processors} \to \text{materializations} \\ &\text{canonical events} \to \text{batch oracle} \to \text{comparison.} \end{aligned}$$

The oracle groups canonical records by key and chooses the maximum version before fault injection. It calls no processor transition. A shared whole-input validator rejects conflicting identities and key/version payloads. Therefore principal conflict rejection is an input-contract check, not measured online anomaly detection.

Arrival replay overwrites on each arrival. ID-only replay stores immutable ID payloads and suppresses a previously seen ID, otherwise overwriting. Version-aware replay also rejects lower-version state updates and conflicting equal-version payloads. The principal ledger is unbounded. These are reference materializers, not transactional sinks; final-state agreement says nothing about external side-effect multiplicity.

3.1 Simulated interruption

All processors serialize state plus the identity ledger at a common earlier source offset, process farther, discard uncommitted work and restore the logical snapshot. The suffix is replayed from the checkpoint. Redelivery workloads then deliver the entire input again. Counts separate attempted work before interruption, lost attempts, restart deliveries and redelivery deliveries. A snapshot is a JSON round trip, not a file write, fsync or atomic source/sink commit. Restoring an older offset and state is a modeling assumption, not a demonstrated crash-durability mechanism.

3.2 Finite identity retention

A separate online experiment uses FIFO ID capacities 0, 4, 16 and unlimited; duplicate hits do not refresh the FIFO. Materialized state and tombstones are still unbounded. After sufficient intervening IDs, the first ID is redelivered exactly, followed by a conflicting payload under that ID. This order is significant: a stale replay can reinsert an evicted identity and enable subsequent conflict detection. The experiment must not be interpreted as a general detection guarantee for expired IDs.

Whole-stream prevalidation is deliberately omitted from this separate question; otherwise the finite-ledger distinction would be concealed. The principal and bounded results cannot be pooled as detection accuracy. Neither experiment establishes exactly-once execution or a safe retention horizon.

4 Experiment and observed outcomes

The repository protocol was committed before implementation and measured execution. The principal matrix crosses 8/64 keys, three seeds, seven conditions and three strategies: 126 rows. Conditions are ordered, shuffled, duplicate delivery, delete/reinsert, conflict, checkpoint interruption and redelivery. Duplicate delivery also shuffles records, so it is a compound ordering/duplication condition rather than an isolated duplicate-treatment estimate. Five fresh timed executions are retained per row, with rotating policy order.

The valid canonical stream has four versions per key, including a version-3 deletion and version-4 reinsert. Correctness compares entire final state to the independent oracle. It also records wrong-key counts, decreasing-version writes, redelivery stability and recovery counts. Invalid-input rows have null correctness values; they are neither correct states nor technical failures.

Policy	Correct	Wrong	Rejected
Arrival	6	30	6
ID-only	6	30	6
Version	36	0	6

In the retained run, each baseline had 30 decreasing-version invariant failures; version-aware replay had none. All tested redelivery-state stability checks passed, including the incorrect baselines. This negative diagnostic finding is central: redelivery stability alone cannot establish state correctness.

The separate bounded matrix has 36 rows. For all three seeds, finite ID-only ledgers changed state on old redelivery. Version-aware replay preserved state at every capacity tested, but capacities 0/4/16 did not recognize the expired exact duplicate. At capacity 0, a conflicting stale payload was not detected. Capacities 4/16 detected the subsequent conflict after the exact replay had reinserted the ID. Unlimited ID memory recognized the duplicate and detected the conflict. This result is sensitive to probe order and retained tombstones; broader guarantees are unwarranted.

Timing includes validation, copying, serialization and replay but excludes generation, oracle/scoring and disk output. Observed principal median ranges were 0.020040–20.123117 ms (arrival), 0.021011–25.376994 ms (ID-only), and 0.021251–26.441142 ms (version-aware) on Python 3.12.14. Early-rejected inputs perform less work and must not be pooled into speedup claims. These are local measurements, not production latency, throughput or performance evidence.

5 Demonstration interactions

The audience first inspects the stale-delete counterexample and predicts each policy's outcome. Next, the presenter runs the existing seeded JSON demo and compares baseline state, oracle state and wrong-key evidence. Checkpoint/interruption controls expose lost attempts and replay counts. Finally, the bounded CSV contrasts correct final state with forgotten identities and missed conflicts. This is currently a command-line and artifact interaction. A graphical side-by-side viewer is proposed work, not an implemented feature.

A five-minute path assigns one minute each to event semantics, counterexample, checkpoint restoration, finite retention, and caveats. The demonstration must show both failure and containment rather than selecting only favorable examples.

6 Related work and open review questions

MillWheel describes persistent state and fault-tolerant stream processing [1]. The Dataflow Model describes correctness, latency and cost concerns for unordered input [2]. Their official VLDB PDFs were retrieved and read; the latter DOI metadata was also checked through Crossref. Our local version comparison does not reproduce these systems or their reported deployments. Questions for a fuller literature review include CDC source ordering contracts, transactional sink coupling, safe identity expiry, tombstone garbage collection, and overlap with teaching and differential-testing artifacts. This focused reading is not an exhaustive novelty assessment.

7 Limitations and reproducibility

No durable checkpoint, external CDC connection, real crash, concurrency or distributed scheduler is tested. Equal versions cannot resolve genuinely ambiguous payloads. Final-state equality is not side-effect correctness. State remains unbounded in the finite-ID experiment. Three deterministic seeds are not a population model, and measurements from one host require independent replication.

From the repository root, run the full unittest suite and V4/V5 validation scripts, then `python research/cdc_lab/run.py --output /tmp/new-run`. The directory must be new. JSON/CSV artifacts preserve seeds, source commit, tracked dirty status, Python/platform, workload/protocol/runner hashes, counters and all timing samples. SHA256 manifests permit integrity checking. The retained run used source commit `8e91b4b` with no tracked modifications. Protocol and implementation commits are separate. Historical artifacts are unchanged. No paper has been submitted by this task. Official 2027 venue requirements remain unverified because its website returned HTTP 403; this four-page review layout is not a compliance claim.

References

[1] T. Akidau et al. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *PVLDB* 6(11), 1033–1044, 2013.

[2] T. Akidau et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *PVLDB* 8(12), 1792–1803, 2015. DOI: 10.14778/2824032.2824076.